END-TO-END MLOPS PROJECT REPORT

HEART DISEASE RISK CLASSIFICATION API

Student: Santhosh Mohan
BITS ID: 2025ae05426
Repository: https://github.com/Explore-AI-Engg/MLOPS
Dataset: UCI Cleveland Heart Disease dataset
Project status: Local pipeline and GKE deployment verified; GitHub Actions success evidence is pending the next workflow run

This report is written from the implementation in this repository. The supplied reference report was used only to understand a possible academic report structure. Its wording, screenshots, code, and numerical results are not used here.

ABSTRACT

This project implements a reproducible machine-learning service for binary heart-disease risk classification. The solution begins with scripted acquisition of the UCI dataset, validates and transforms the data, compares Logistic Regression and Random Forest models, tracks experiments with MLflow, and persists the selected preprocessing-and-model pipeline as a Joblib artifact. A FastAPI service exposes health, prediction, and Prometheus metrics endpoints. Automated tests and linting run in GitHub Actions, while a Docker image packages the API and trained artifact for deployment to Google Kubernetes Engine (GKE).

The project deliberately keeps preprocessing inside the persisted scikit-learn pipeline. This prevents training-serving skew: the same imputers, scaling rules, and categorical encoding learned during training are used for API requests. Operational logs exclude patient feature values and record only request metadata, status, and latency.

1. PROJECT OBJECTIVE AND ARCHITECTURE

The objective is to demonstrate the complete lifecycle rather than only achieve a high offline score:

1. Acquire and validate data.
2. Explore data quality, distributions, correlation, and class balance.
3. Train at least two classifiers using a reproducible pipeline.
4. Select a model using cross-validation and clinically relevant metrics.
5. Track parameters, metrics, plots, and model artifacts.
6. Test data processing, modeling, and API behavior.
7. Package the service in Docker.
8. Deploy the image to Kubernetes.

9. Expose health, prediction, logs, and metrics for operation.

The logical flow is:

text
UCI dataset
- download_data.py
- prepare_data.py
- eda.py
- train.py + MLflow
- artifacts/model.joblib
- Docker image
- GHCR
- GKE Deployment
- LoadBalancer Service
- /health, /predict, /metrics

The repository separates responsibilities:

- app/: reusable application and preprocessing code.
- scripts/: repeatable data, EDA, training, and smoke-test entry points.
- tests/: automated tests.
- artifacts/: persisted models and metadata.
- outputs/eda/: generated visualizations.
- deployment/: Kubernetes resources.
- .github/workflows/: CI and deployment automation.

2. DATA ACQUISITION AND PREPARATION

The download script obtains the public UCI archive from the configured source URL and writes a validated raw CSV. The Cleveland dataset contains 303 observations and 13 predictor columns. The original target uses values from 0 to 4; this project maps 0 to no observed disease and values 1 through 4 to disease present.

The data preparation process:

1. Checks the expected columns and row structure.
2. Converts the target to a binary value.
3. Represents missing values explicitly.
4. Writes the processed CSV under data/processed/.
5. Leaves the raw download available for reproducibility.

The input features are:

age, sex, cp, trestbps, chol, fbs, restecg, thalach, exang, oldpeak, slope, ca, and thal.

The dataset is small, so the project uses stratified splitting and cross-validation. This preserves the approximate class ratio in training and validation partitions. The model should not be interpreted as a clinical diagnostic system: the dataset is limited in size and scope, and a real medical application would require external validation, clinical review, calibration, and regulatory controls.

3. EXPLORATORY DATA ANALYSIS

The EDA script creates:

- Feature distribution plots to identify scale differences and unusual values.
- A correlation heatmap to inspect linear relationships.
- A class-balance chart to show target proportions.
- A JSON summary for repeatable numerical inspection.

EDA informs the design without becoming a source of leakage. The target is not used to fit imputers or scalers. The generated files are:

- outputs/eda/feature_distributions.png
- outputs/eda/correlation_heatmap.png
- outputs/eda/class_balance.png
- outputs/eda/summary.json

These outputs should be included in the final submission under evidence/ or linked from the report after capturing the final repository state.

4. FEATURE ENGINEERING AND MODEL DEVELOPMENT

The model pipeline divides variables into numeric and categorical groups. Numeric columns use median imputation followed by standard scaling. Categorical columns use most-frequent imputation followed by one-hot encoding with unknown-category handling. The classifier is attached to the same ColumnTransformer pipeline.

This design was selected for three reasons:

1. Median imputation is robust to a small number of missing clinical measurements.
2. Scaling supports Logistic Regression optimization and prevents large-unit variables from dominating the regularized objective.
3. One-hot encoding makes categorical values usable by scikit-learn estimators while

handle_unknown="ignore" protects the API from an unseen category.

Two models are trained:

- Logistic Regression: a transparent linear baseline with stable probabilities.
- Random Forest: a nonlinear ensemble that can capture interactions and threshold effects.

Five-fold stratified cross-validation is used for comparison. The primary selection metric is cross-validation ROC-AUC because it evaluates ranking quality across thresholds. Accuracy, precision, and recall are also reported because they describe different operational trade-offs.

5. RESULTS

The selected model in the current artifact is Logistic Regression. The recorded results are:

Evaluation	Accuracy	Precision	Recall	ROC-AUC
5-fold cross-validation	0.8471	0.8768	0.7834	0.9025
Holdout test set	0.8852	0.8387	0.9286	0.9665

The holdout recall is higher than the cross-validation recall. This is expected from a single finite split and is why the cross-validation result is used for model selection rather than relying on one holdout number. These results demonstrate a functioning educational classifier; they do not establish clinical performance.

The artifact metadata records the model name, MLflow run identifier, dataset hash, feature count, selection metric, and evaluation metrics. This makes the deployed model traceable to a specific training input and experiment.

6. MLFLOW EXPERIMENT TRACKING AND REPRODUCIBILITY

Each training run logs:

- Model and preprocessing parameters.
- Cross-validation and holdout metrics.
- Evaluation plots.
- Dataset hash.
- The selected model artifact and metadata.

The local file tracking store is mlruns/. A dataset hash is recorded so a future run can identify whether the input data changed. The reproducible training sequence is:

bash
python -m scripts.download_data
python -m scripts.prepare_data
python -m scripts.eda
python -m scripts.train --tracking-uri file:./mlruns

The model is saved as artifacts/model.joblib. Saving the complete pipeline, rather than only the classifier, is essential because inference must repeat imputation, scaling, and encoding exactly as training did.

7. API IMPLEMENTATION

The FastAPI service provides:

- GET /health: returns 200 when the model artifact is loaded and 503 when it is unavailable.
- POST /predict: validates one 13-feature JSON payload and returns a binary prediction, confidence, and model version.
- GET /metrics: returns Prometheus-compatible request counters and latency histograms.

The Pydantic request model rejects unknown fields and applies reasonable range checks. The API middleware records HTTP method, path, status, and latency. It intentionally does not log raw patient features.

Example local request:

bash
curl -X POST http://127.0.0.1:8000/predict \
  -H 'Content-Type: application/json' \
  -d '{"age":63,"sex":1,"cp":3,"trestbps":145,"chol":233,"fbs":1,"restecg":0,"thalach":150,"
exang":0,"oldpeak":2.3,"slope":0,"ca":0,"thal":1}'

For local operation:

bash
uvicorn app.api:app --host 0.0.0.0 --port 8000
curl http://127.0.0.1:8000/health
curl http://127.0.0.1:8000/metrics

8. TESTING AND CONTINUOUS INTEGRATION

The test suite covers:

- Required data columns and target conversion.
- Preprocessor construction and transformations.
- Model training and prediction behavior.
- API health and prediction responses.

The CI workflow performs linting, downloads data, prepares data, generates EDA outputs, trains the model, runs Pytest, builds the Docker image, and uploads artifacts. This ordering catches failures before a deployment image is published.

Recommended local validation:

bash
python -m ruff check app scripts tests
python -m pytest -q
docker build -t heart-disease-api:local .

9. CONTAINERIZATION

The Docker image installs the pinned project dependencies, copies the app/ package and trained artifacts/, and starts Uvicorn on port 8000. The model artifact must be present in the image; otherwise the service correctly reports 503 from /health rather than claiming readiness.

The deployment workflow therefore trains and verifies artifacts/model.joblib before the Docker build. The image name is lowercase because Docker and GHCR reject uppercase repository components:

text
ghcr.io/explore-ai-engg/heart-disease-api

Before deploying a new image, verify the artifact directly:

bash
docker run --rm --entrypoint sh heart-disease-api:local \
  -c 'test -f /app/artifacts/model.joblib && ls -lh /app/artifacts'

10. KUBERNETES AND DEPLOYMENT STATUS

The Kubernetes manifest defines a Deployment named heart-disease-api and a Service for external access. The Deployment uses a readiness check against /health, so traffic is not sent to a pod before the model is loaded.

An earlier GKE rollout returned 503 from /health and entered CrashLoopBackOff. That was a real failure signal and was not used as successful evidence. Investigation showed that the single e2-small node could not schedule the replacement pod because the previous resource requests were too large for available memory. The deployment was corrected by using a Recreate strategy, lower resource requests, and a startup probe. The corrected rollout completed successfully.

1. The workflow ran for the branch that was pushed.
2. The build context contains artifacts/model.joblib.
3. The pushed image digest is the image used by the Deployment.
4. The pod can read /app/artifacts/model.joblib.
5. The pod logs identify any model-loading exception.
6. The Deployment reaches 1/1 ready replicas before the external URL is tested.

The deployment workflow has been updated to run on both main and master, to train before building, and to print diagnostics only when a rollout step fails. A public API URL should be added to this report only after all of the following succeed:

bash
kubectl rollout status deployment/heart-disease-api --timeout=180s
kubectl get service heart-disease-api
curl http://EXTERNAL_IP/health
curl -X POST http://EXTERNAL_IP/predict -H 'Content-Type: application/json' -d '...'

The verified public service is:

text
http://104.154.36.8:8000

Observed Kubernetes state after the fix:

text
deployment/heart-disease-api: 1/1 available
pod: heart-disease-api-6d655b99b7-lcpfl 1/1 Running
service: LoadBalancer EXTERNAL-IP 104.154.36.8

Verified endpoint responses:

text
GET /health - 200
{"status":"ready","model":"c235876a9c844214ab4a68fd3c0e07db"}

POST /predict - 200
{"prediction":0,"confidence":0.814063,"model_version":"c235876a9c844214ab4a68fd3c0e07db"}

GET /metrics - 200
Prometheus metrics included heart_disease_api_requests_total and heart_disease_api_request_latency_seconds.

The earlier failed rollout may be included in a troubleshooting section, but it must not be presented as successful deployment evidence.

11. MONITORING, LOGGING, PRIVACY, AND SECURITY

Prometheus counters measure request totals by endpoint and status. A latency histogram records request duration. These signals can be scraped by Prometheus and visualized in Grafana, or inspected directly through /metrics.

Operational logging includes method, route, status, and latency. It does not include the JSON body. This reduces accidental exposure of health information in logs. Production hardening would additionally require authentication, TLS, network policy, secret management, rate limiting, audit controls, and a formal privacy review.

GitHub Actions authenticates to Google Cloud using Workload Identity Federation rather than a long-lived service-account key. The provider secret must contain the full provider resource name without a https:// or //iam.googleapis.com/ prefix.

12. REPRODUCTION GUIDE

bash
```
git clone https://github.com/Explore-AI-Engg/MLOPS.git
cd heart-disease-mlops
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements.txt
python -m scripts.download_data
python -m scripts.prepare_data
python -m scripts.eda
```

```
python -m scripts.train --tracking-uri file:./mlruns
python -m pytest -q
uvicorn app.api:app --host 0.0.0.0 --port 8000
```

The single learning notebook, when present under notebooks/, explains the same lifecycle interactively. The scripts remain the source of truth for CI/CD because they are deterministic command-line entry points.

13. EVIDENCE CHECKLIST

The final submission should contain original screenshots with readable timestamps or workflow links:

1. Dataset download and processed row count.
2. EDA plots.
3. MLflow run page or local MLflow artifact listing.
4. Test and lint success.
5. Docker build and local /health response.
6. Local /predict response.
7. GitHub Actions CI success.
8. Successful Kubernetes rollout and 1/1 ready pod.
9. Successful public /health, /predict, and /metrics responses.
10. Short video link showing the end-to-end flow.

The repository?s evidence/README.md records the expected filenames and prevents accidental submission of fabricated evidence.

14. LIMITATIONS AND FUTURE IMPROVEMENTS

The dataset is small, historical, and not sufficient for clinical deployment. Future work should add external validation, calibration, subgroup analysis, threshold selection based on clinical cost, drift monitoring, model registry promotion rules, authenticated API access, TLS, and managed observability.

The current project demonstrates the engineering lifecycle and reproducibility controls expected in an MLOps assignment. It should not be represented as a medical device or used to make patient-care decisions.

CONCLUSION

This repository demonstrates an end-to-end, reproducible ML workflow: scripted data acquisition, explicit preprocessing, model comparison, MLflow tracking, automated tests,

Docker packaging, Kubernetes deployment configuration, and operational metrics. The local training and API components are verified. Cloud deployment claims remain conditional on a successful GKE rollout and direct endpoint verification; this distinction keeps the submission technically accurate and auditable.